

CSE 4403: Algorithms

Lecture05+06

Graph-2: Shortest Path Algorithms

Anika Farzana

Junior Lecturer

Department of Computer Science & Engineering

Islamic University of Technology, Dhaka, Bangladesh

Email: anikafarzana@iut-dhaka.edu

Shortest Path Algorithms

Given a **weighted directed/undirected** graph and a source vertex, we need to find the shortest path distances from the source vertex to all other vertices in the graph.

Shortest Path Algorithms

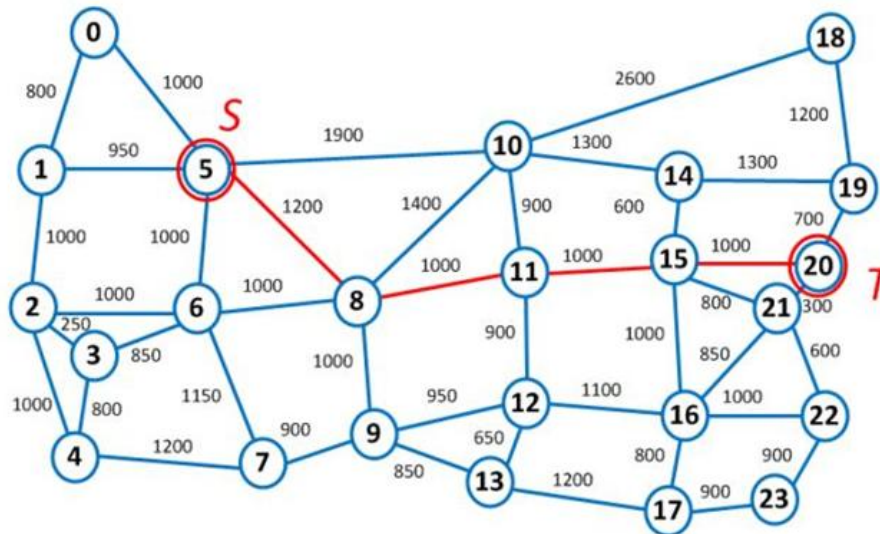
Given a **weighted directed/undirected** graph and a source vertex, we need to find the shortest path distances from the source vertex to all other vertices in the graph.

Two well-known single source shortest path algorithms:

1. Dijkstra's Algorithm (uses greedy approach)
2. BellmanFord Algorithm (uses dp approach)

Dijkstra's Algorithm

- Dijkstra's algorithm is a graph search algorithm that solves single-source shortest path for a graph with **non-negative edge weights**.
- Widely used in networking protocols such as Open Shortest Path First (OSPF).



Dijkstra's Algorithm: Pseudocode

DIJKSTRA(G, source):

for each vertex v in G :

$\text{dist}[v] \leftarrow \infty$

$\text{vis}[v] \leftarrow 0$

$\text{dist}[\text{source}] \leftarrow 0$

create a priority queue PQ where priority will be on $\text{dist}[]$

PQ.push($\{0, \text{source}\}$)

while PQ is not empty:

$\text{node} \leftarrow \text{PQ.pop}()$

$d \leftarrow \text{node.first}$

$u \leftarrow \text{node.second}$

 if $\text{vis}[u] == 1$: continue

$\text{vis}[u] \leftarrow 1$

 for each neighbor v of u :

 if $\text{vis}[v] == 1$: continue

 if $d + \text{weight}(u, v) < \text{dist}[v]$:

$\text{dist}[v] \leftarrow d + \text{weight}(u, v)$

 PQ.push($\{\text{dist}[v], v\}$)

return $\text{dist}[]$

Dijkstra's Algorithm

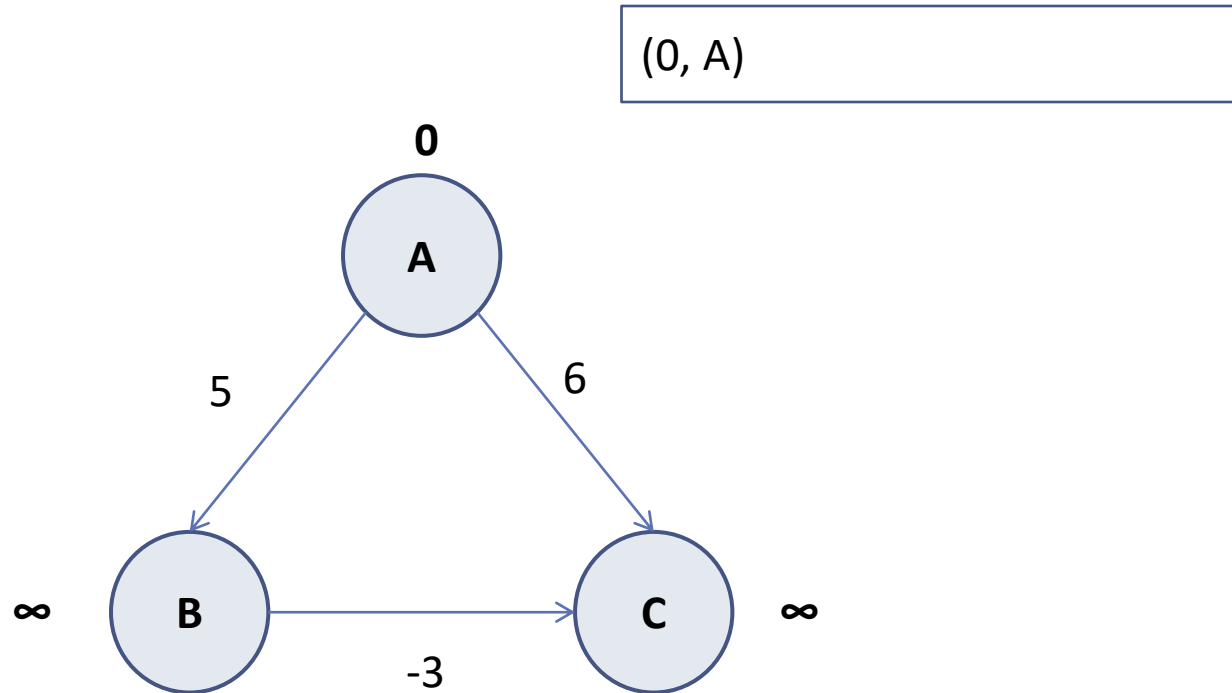
Time Complexity

Time Complexity: $O((V+E)*\log V)$

Space Complexity: $O(V+E)$

Dijkstra's Algorithm

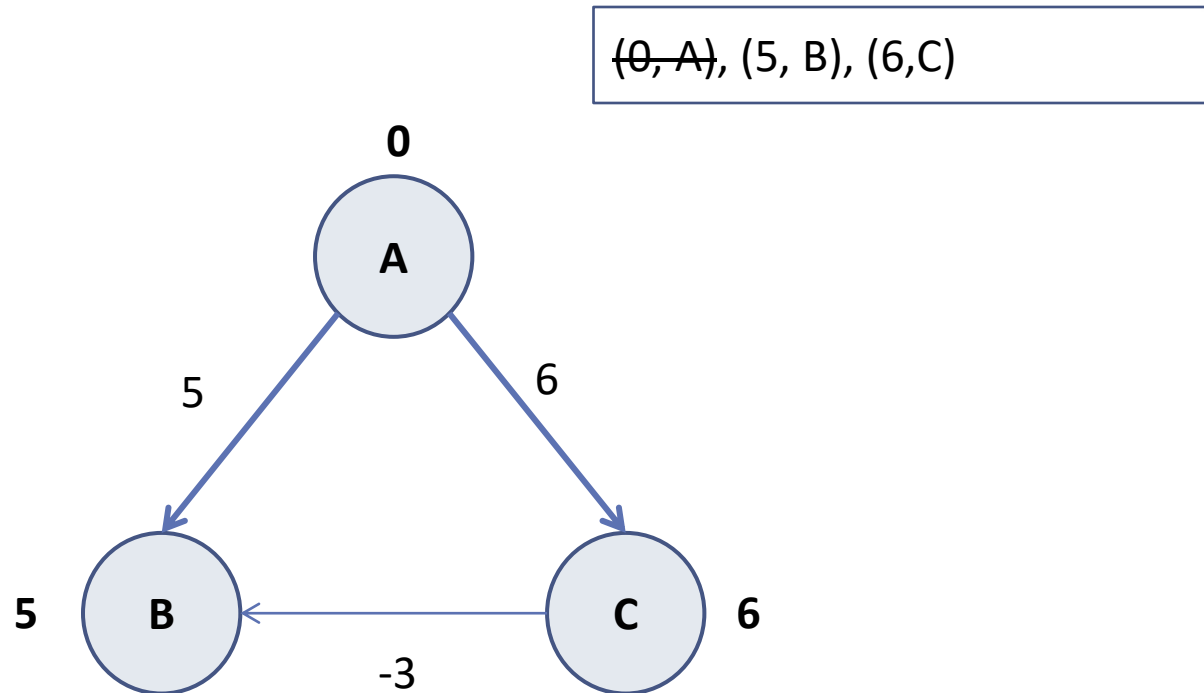
Problem with negative edges



A is the source and pushed into the queue

Dijkstra's Algorithm

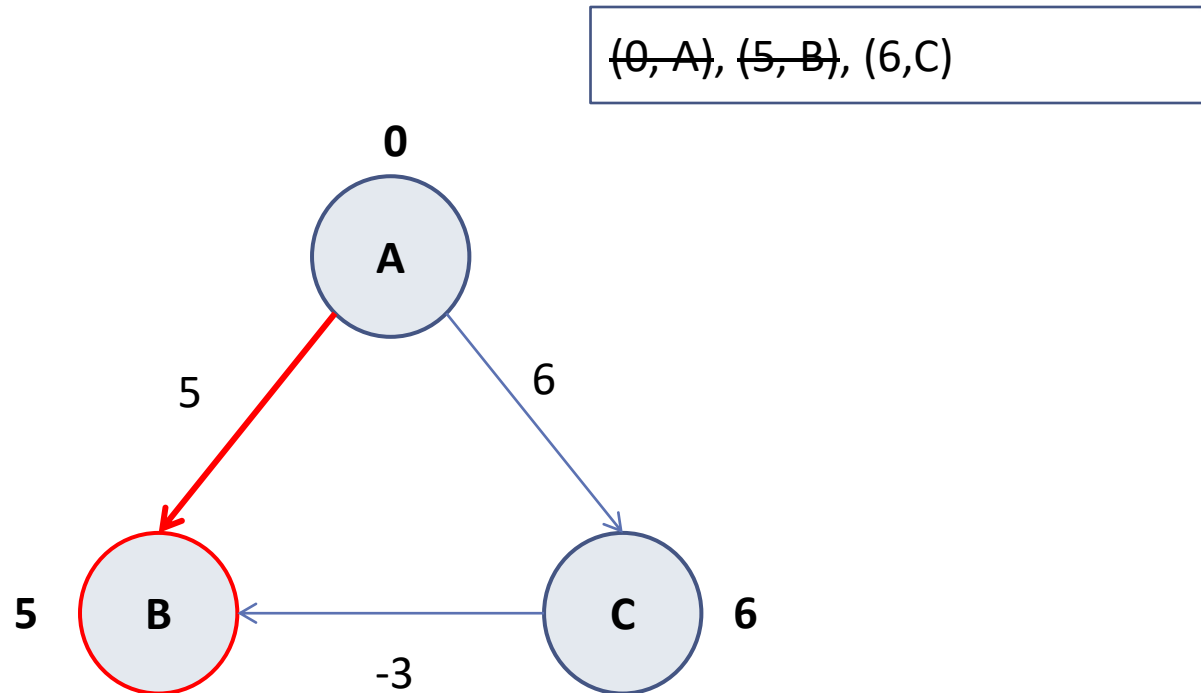
Problem with negative edges



The distance of the neighbors of A are updated and pushed into the queue.

Dijkstra's Algorithm

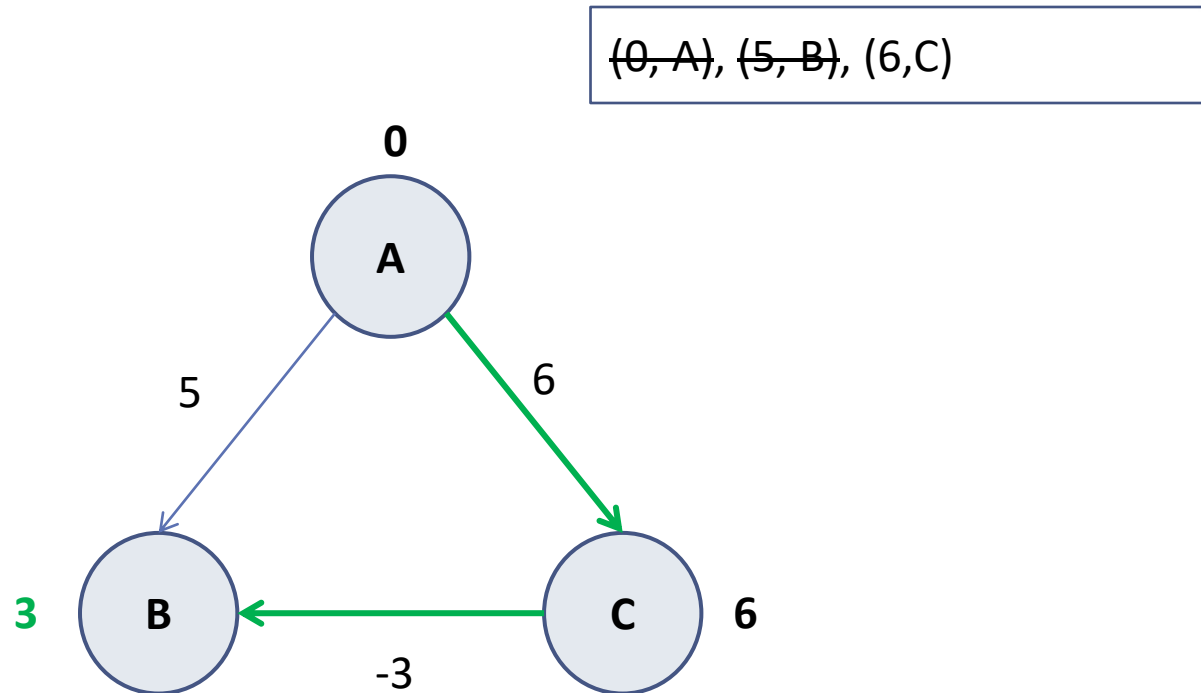
Problem with negative edges



The distance of B is 5 and will not change anymore as it is popped from the queue before C and marked as visited

Dijkstra's Algorithm

Problem with negative edges

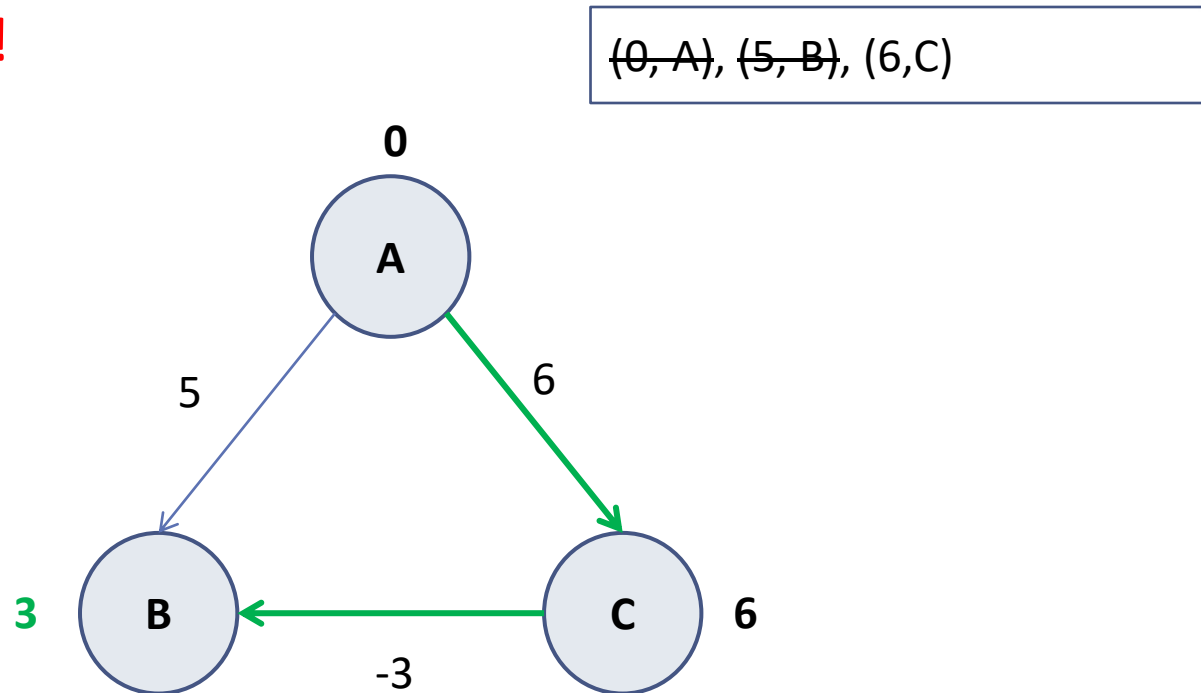


But the optimal distance of B through the path A→C→B is $6 + (-3) = 3$

Dijkstra's Algorithm

Problem with negative edges

Dijkstra failed!



But the optimal distance of B through the path A→C→B is $6 + (-3) = 3$

Bellman-Ford Algorithm

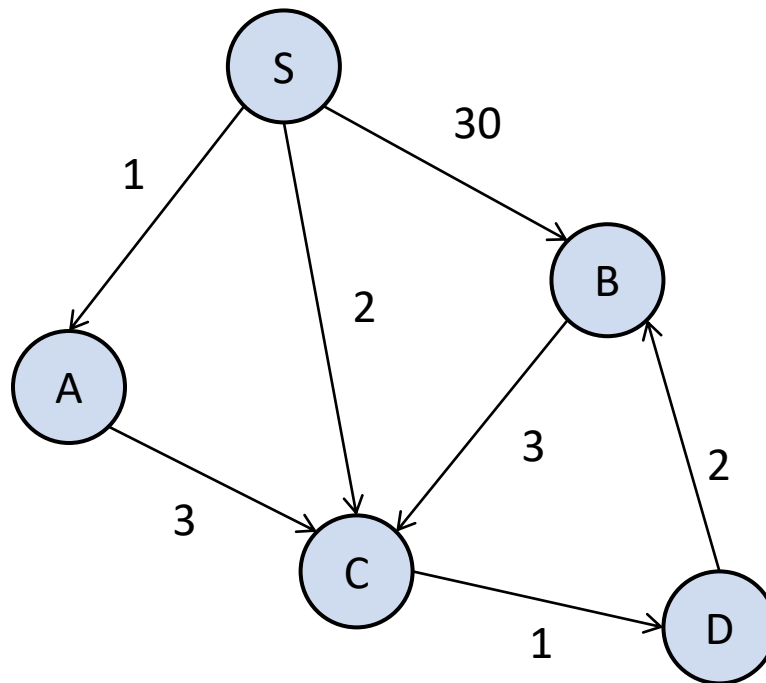
- Bellman-Ford Algorithm is single-source shortest path algorithm that computes the shortest distances from a source node to all other nodes in a weighted graph.
- Unlike Dijkstra, it can **handle negative edge weights** and **detect negative edge cycles**.
- Used in the **RIP(Routing Information Protocol)**, where the routers periodically share their routing tables with the neighbor routers to update their shortest paths.
- Routers do **not** need to know the complete network topology.
- “If my neighbor knows a better route, I update mine.”

Bellman-Ford Algorithm

Code with no negative cycle detection

```
# Relaxation of all the edges V times, not (V - 1) as we
# need one additional relaxation to detect negative cycle
for i in range(V):
    for edge in edges:
        u, v, wt = edge
        if dist[u] != 100000000 and dist[u] + wt < dist[v]:
            # Update shortest distance to node v
            dist[v] = dist[u] + wt
return dist
```

Bellman-Ford Algorithm

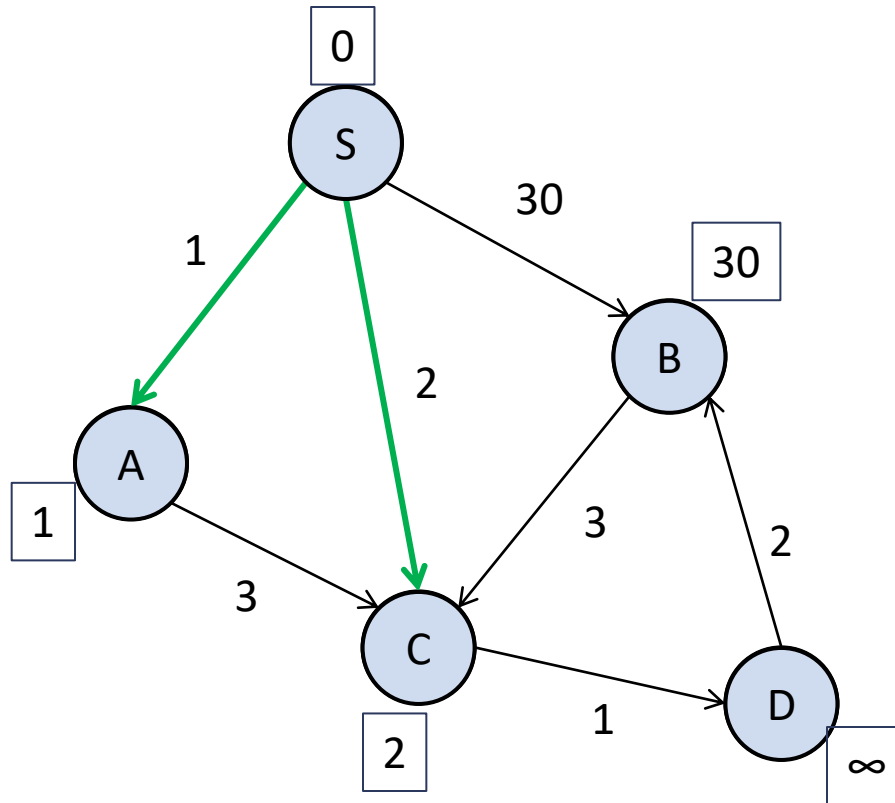


Edges

S->A
D->B
S->B
C->D
B->C
S->C
B->C

Bellman-Ford Algorithm

After 1st pass:

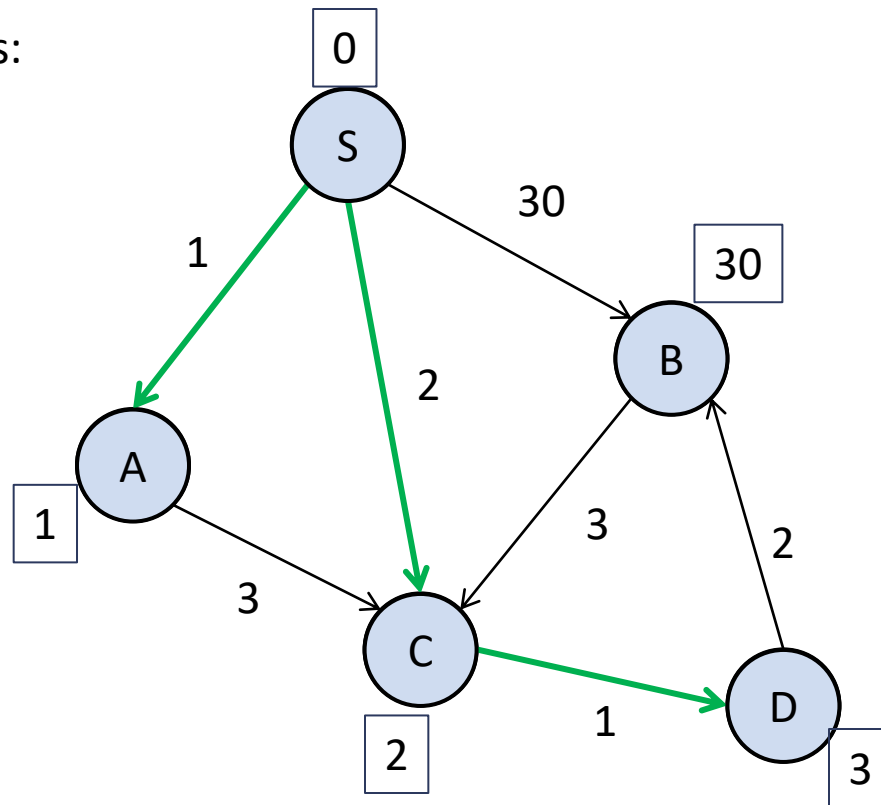


Edges

S->A
D->B
S->B
C->D
B->C
S->C
B->C

Bellman-Ford Algorithm

After 2nd pass:

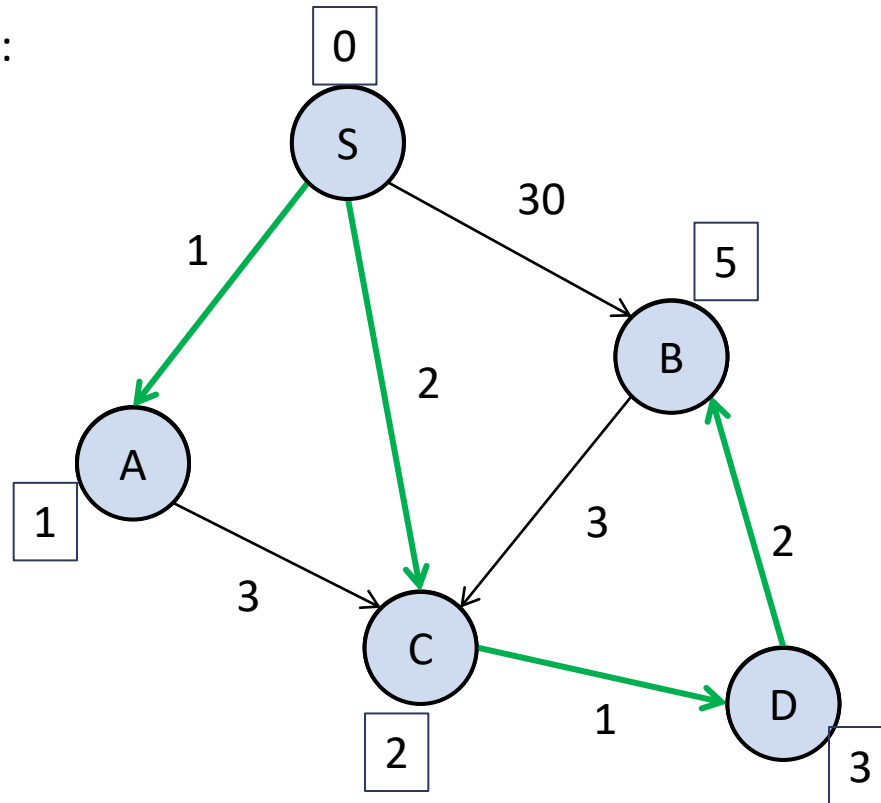


Edges

S->A
D->B
S->B
C->D
B->C
S->C
B->C

Bellman-Ford Algorithm

After 3rd pass:

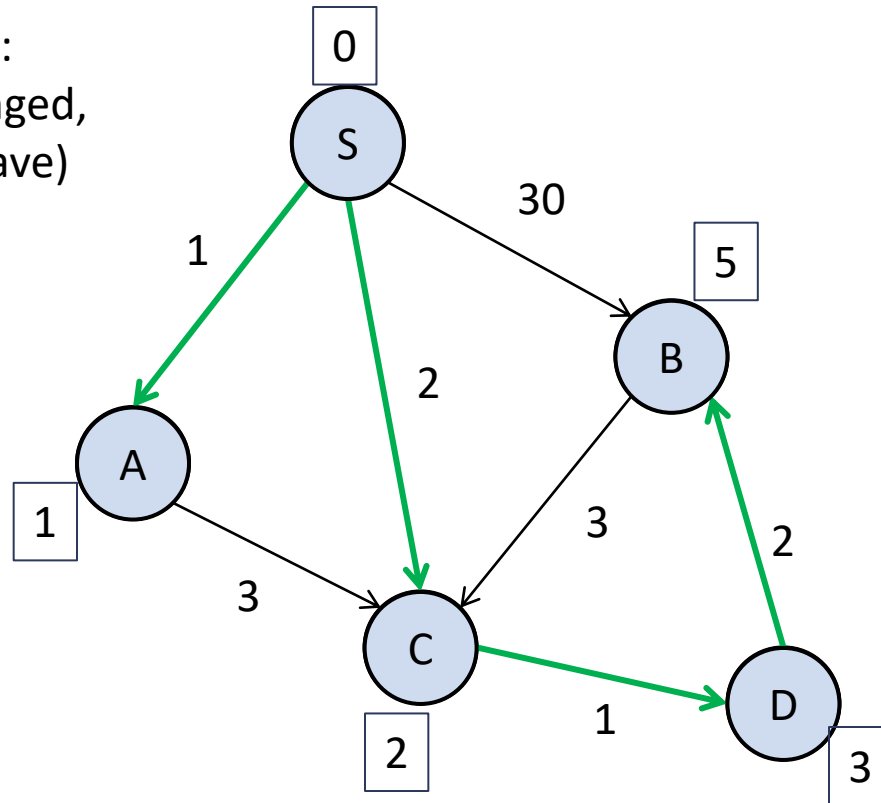


Edges

S->A
D->B
S->B
C->D
B->C
S->C
B->C

Bellman-Ford Algorithm

After 4th pass:
(Nothing changed,
but it could have)

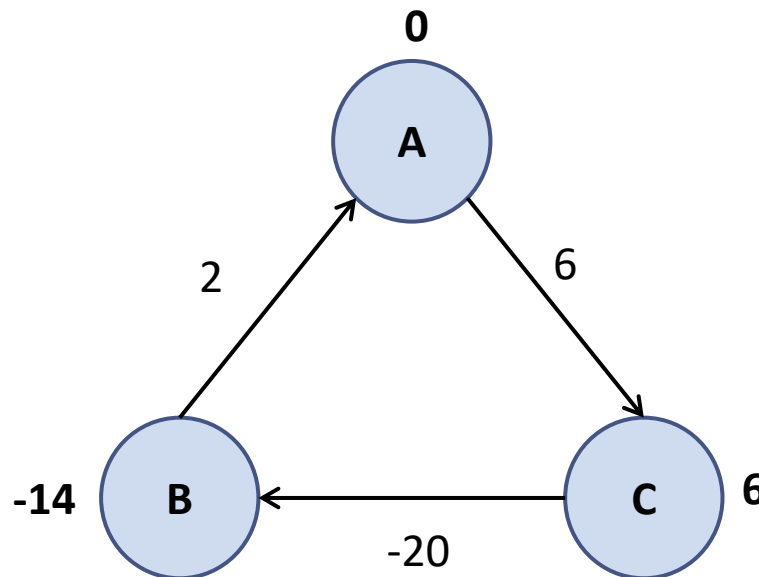


Edges

S->A
D->B
S->B
C->D
B->C
S->C
B->C

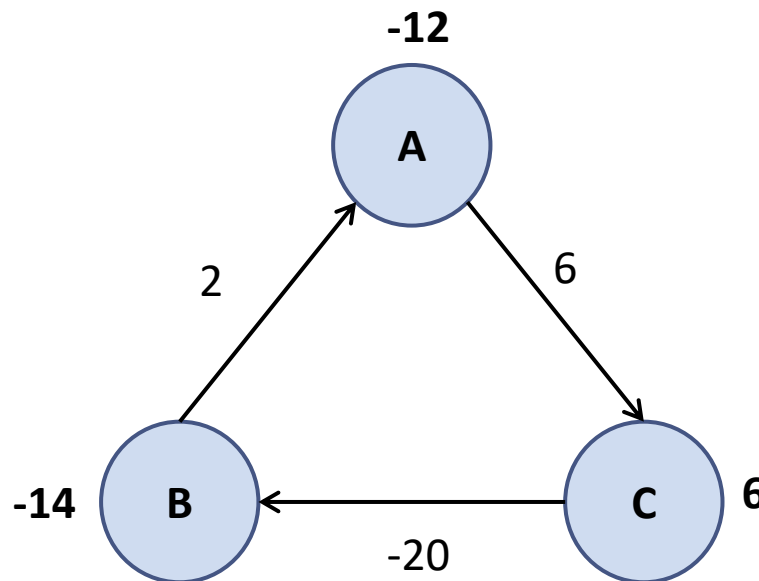
Negative Weight Cycle

In the presence of negative weight cycle in the graph, the shortest path **doesn't exist** because with each traversal of the cycle shortest path keeps **decreasing**.



Negative Weight Cycle

In the presence of negative weight cycle in the graph, the shortest path **doesn't exist** because with each traversal of the cycle shortest path keeps **decreasing**.



Bellman-Ford Algorithm

Code with negative cycle detection

```
# Relaxation of all the edges V times, not (V - 1) as we
# need one additional relaxation to detect negative cycle
for i in range(V):
    for edge in edges:
        u, v, wt = edge
        if dist[u] != 100000000 and dist[u] + wt < dist[v]:

            # If this is the Vth relaxation, then there
            # is a negative cycle
            if i == V - 1:
                return [-1]

            # Update shortest distance to node v
            dist[v] = dist[u] + wt
return dist
```

Bellman-Ford Algorithm

Time and Space Complexity

Time Complexity: $O(V \cdot E)$

Space Complexity: $O(V)$

Dijkstra vs Bellman-Ford

Comparison

Feature	Dijkstra	Bellman-Ford
Negative edge handling	No	Yes
Negative cycle detection	No	Yes
Faster	Yes	No
Greedy	Yes	No
Relax all edges repeatedly	No	Yes